

FreelanceHub — App Functionality Explanation

This architecture separates features into different apps.
That makes the project:

- clean
- scalable
- professional
- easier to maintain

Think of each app as a separate department of a company.

Overall Platform Scenario

Client

A company/person who wants work done.

Example:

“I need a website for my business.”

Client can:

- create projects
 - receive proposals
 - hire freelancers
 - review freelancers
-

Freelancer

A developer/designer who wants work.

Example:

“I can build your website for \$300.”

Freelancer can:

- browse projects
 - apply to projects
 - manage profile
 - receive reviews
-

1. **accounts/** App

Purpose

Handles authentication and user system.

Features

Client

- Register account
 - Login
 - Logout
 - Email verification
-

Freelancer

- Register account
- Login
- Logout
- Password reset

Scenario

Example

Shazzad wants to join FreelanceHub.

He:

1. Creates account
2. Chooses role = Freelancer
3. Verifies email
4. Logs into dashboard

Now he can apply for jobs.

Main Responsibilities

Authentication

User roles

Login session

Email verification

Password management

Models Example

User

- username
 - email
 - password
 - role
 - is_verified
-

2. profiles/ App

Purpose

Stores extra user information.

Features

Freelancer

- Upload profile image
 - Add skills
 - Add bio
 - Add portfolio links
-

Client

- Company details
 - Profile picture
 - About company
-

Scenario

A client visits Shazzad's profile and sees:

- Python
- Django
- DRF
- Portfolio links
- Previous ratings

This helps the client trust the freelancer.

Main Responsibilities

Profile information

Profile image

Skills

Social links

Portfolio

Models Example

Profile

- user

- image

- bio

- skills

- github

- linkedin

3. projects/ App

Purpose

Main business logic of the platform.

This is the HEART of the system.

Features

Client

- Post project
 - Edit project
 - Delete project
 - View proposals
-

Freelancer

- Browse projects
 - View details
 - Search projects
 - Filter projects
-

Scenario

A client posts:

Need E-commerce Website

Budget: \$500

Deadline: 10 days

Freelancers can now apply.

Main Responsibilities

Project CRUD

Project listing

Search

Filtering

Sorting

Models Example

Project

- title
 - description
 - budget
 - deadline
 - category
 - client
-

4. proposals/ App

Purpose

Handles freelancer applications.

Features

Freelancer

- Submit proposal
 - Write cover letter
 - Add bid amount
-

Client

- View all proposals
 - Accept proposal
 - Reject proposal
-

Scenario

Shazzad sees a project:

“Build Portfolio Website”

He submits:

I can build this using Django and Bootstrap.

Budget: \$250

Delivery: 5 days

Client reviews proposal and hires him.

Main Responsibilities

Project applications

Proposal management

Hiring workflow

Bid system

Models Example

Proposal

- project

- freelancer

- cover_letter

- bid_amount

- status

5. reviews/ App

Purpose

Creates trust in the platform.

Features

Client

- Rate freelancer
 - Write feedback
-

Freelancer

- Receive ratings
 - Build reputation
-

Scenario

After project completion:

Client gives:

- Rating: 5★
- Comment:
“Excellent communication and fast delivery.”

Now Shazzad’s profile becomes stronger.

Main Responsibilities

Ratings

Feedback

Trust system

Reputation

Models Example

Review

- project
 - freelancer
 - client
 - rating
 - comment
-

6. categories/ App

Purpose

Organizes projects into groups.

Features

Categories Example

Web Development
Mobile App
UI/UX Design
Machine Learning
Graphic Design

Scenario

A freelancer only wants:

“Web Development”

He filters projects by category.

Now he sees only relevant jobs.

Main Responsibilities

Project categorization

Filtering support

Search organization

Models Example

Category

- name

7. **core/** App

Purpose

Common/shared functionality.

Think of this as:

“global website system”

Features

Homepage

About page

Contact page

Navbar
Footer
Dashboard redirects
Global utilities

Scenario

When anyone visits:

/

the homepage comes from `core`.

Main Responsibilities

Common templates
Shared views
Global context
Utilities

Other Important Folders

`templates/`

Contains HTML files.

Example:

base.html
home.html
login.html
project_list.html

static/

Contains:

- CSS
- JS
- Images

Example:

style.css
main.js
logo.png

media/

Stores uploaded files.

Example:

- profile images
 - project attachments
-

Real Workflow of Platform

Full Scenario

Step 1 — Client Registers

Using:

accounts app

Step 2 — Client Creates Profile

Using:

profiles app

Step 3 — Client Posts Project

Using:

projects app

Step 4 — Freelancer Browses Projects

Using:

projects + categories

Step 5 — Freelancer Sends Proposal

Using:

proposals app

Step 6 — Client Accepts Proposal

Using:

proposals app

Step 7 — Project Finished

Using:

reviews app

Why This Structure Is Professional

Because:

- modular architecture
- scalable
- maintainable
- reusable
- industry-standard approach

Large Django applications follow this pattern.